Python Mastery — 100 Day Roadmap

> Complete Python preparation: Core Python → DSA → Libraries → ML/AI/DL → Web Scraping → Big Tech Interview Prep

---

PHASE 1: Python Fundamentals

Setup & Basics

- [✅] Install Python & Setup environment
- [✅] Hello World with `print()`
- [✅] Using the Python REPL
- [✅] Variables and Assignment
- [✅] Data Types (int, float, str, bool)
- [✅] User Input (`input()`)
- [✅] Basic Math & Operators
- [✅] Type Checking & Conversion (`type()`, `str()`, `int()`, etc.)
- [✅] Simple Expressions
- [ ] Comments & Docstrings
- [ ] Constants & Naming Conventions
- [ ] Walrus operator `:=`

Control Flow

- [✅] If, Else, Elif
- [✅] Comparison & Logical Operators
- [✅] Conditional decisions
- [✅] While Loops
- [✅] For Loops & Ranges
- [✅] `break`, `continue`, `pass`
- [✅] Nested Loops
- [ ] Loop with `else` clause
- [ ] Match-case statements (Python 3.10+)

Functions

- [ ] Writing Functions (`def`, `return`)
- [ ] Parameters & Arguments
- [ ] Default Args, Keyword Args
- [ ] `*args` and `**kwargs`
- [ ] Docstrings
- [ ] Scope (LEGB rule)

- [ ] Lambda functions
- [ ] Recursion basics
- [ ] Closures
- [ ] Decorators
- [ ] Higher-order functions (`map`, `filter`, `reduce`)
- [ ] Generator functions & `yield`

PHASE 2: Data Structures (Built-in)

Lists

- [ ] Creation, Indexing, Slicing
- [ ] List Methods (`append`, `remove`, `pop`, `insert`, `extend`)
- [ ] List Comprehensions
- [ ] Nested Lists
- [ ] `enumerate()`, `zip()`
- [ ] Sorting (`sort`, `sorted`, custom keys)

Tuples

- [ ] Creation & Immutability
- [ ] Tuple unpacking
- [ ] Named tuples
- [ ] Tuple methods

Sets

- [ ] Creation & Operations
- [ ] Set methods (union, intersection, difference)
- [ ] Frozen sets
- [ ] Set comprehensions

Dictionaries

- [ ] Keys & Values
- [ ] Dict methods (`get`, `keys`, `values`, `items`, `update`)
- [ ] Dict comprehensions
- [ ] `defaultdict`, `OrderedDict`, `Counter`
- [ ] Nested dictionaries
- [ ] Merging dictionaries (`|` operator, `**` unpacking)

Strings

- [ ] String methods (`split`, `join`, `strip`, `replace`)

- [ ] String formatting (f-strings, `.format()`, `%`)
- [ ] String slicing & indexing
- [ ] Regular expressions (`re` module)
- [ ] Encoding/Decoding (UTF-8, ASCII)

PHASE 3: Python Practice Programs

Number Programs

- [ ] Counting digits of a number
- [ ] Sum of all digits
- [ ] Reversing digits
- [ ] Factorial (iterative + recursive)
- [ ] Fibonacci series
- [ ] Armstrong number
- [ ] Magic number
- [ ] Palindrome number
- [ ] Prime number check
- [ ] Prime numbers in range
- [ ] Power of a number
- [ ] GCD & LCM
- [ ] Perfect number
- [ ] Strong number
- [ ] Roots of quadratic equation
- [ ] Multiplication table
- [ ] Count down from a given number
- [ ] Number to English word

Array/List Programs

- [ ] Sum of array
- [ ] Max & Min of array
- [ ] Second maximum of array
- [ ] Second largest (distinct)
- [ ] Max-Min difference
- [ ] Search element (find index)
- [ ] Reverse list/array
- [ ] Left rotate by 1
- [ ] Right rotate by k
- [ ] Remove duplicates (unordered)
- [ ] Remove duplicates (keep order)

- [ ] Split positives/negatives
- [ ] Split even/odd by index
- [ ] Separate even/odd elements
- [ ] Sum of even/odd/prime elements
- [ ] Max of even/odd/prime elements
- [ ] Sorting array
- [ ] Merge two lists alternately
- [ ] Common elements in two lists
- [ ] Move zeroes to end
- [ ] Count occurrence of element
- [ ] Find largest element
- [ ] Find sum and average
- [ ] Even and odd lists
- [ ] Rotate list

String Programs

- [ ] Position of character in string
- [ ] Char frequency
- [ ] Vowel count
- [ ] Find vowels
- [ ] Palindrome string
- [ ] Remove duplicates
- [ ] Print without duplication
- [ ] Anagram test
- [ ] Count vowels and consonants
- [ ] Count words in sentence
- [ ] Word frequency counter
- [ ] Reverse words in sentence
- [ ] Longest word in sentence

Matrix Programs

- [ ] Create & print matrix
- [ ] Transpose
- [ ] Diagonal sum
- [ ] Identity check
- [ ] Add/Subtract/Multiply matrices
- [ ] Spiral traversal
- [ ] Rotate matrix 90°

Dictionary Programs

- [ ] Access & count
- [ ] Swap key/value
- [ ] Filter by value
- [ ] Merge dictionaries
- [ ] Phonebook implementation

Tuple/Set Programs

- [ ] Find max/min in tuple
- [ ] Common elements in two sets

PHASE 4: Searching & Sorting Algorithms

Searching

- [ ] Linear Search
- [ ] Binary Search (iterative)
- [ ] Binary Search (recursive)
- [ ] Ternary Search
- [ ] Jump Search
- [ ] Interpolation Search
- [ ] Exponential Search

Sorting

- [ ] Bubble Sort
- [ ] Selection Sort
- [ ] Insertion Sort
- [ ] Merge Sort
- [ ] Quick Sort
- [ ] Counting Sort
- [ ] Radix Sort
- [ ] Heap Sort
- [ ] Bucket Sort
- [ ] Shell Sort
- [ ] Time complexity comparison of all sorts

PHASE 5: Object-Oriented Programming

- [ ] Classes & Objects

- [ ] `__init__` constructor
- [ ] Instance vs Class Attributes
- [ ] Methods (instance, class, static)
- [ ] Encapsulation (private/protected)
- [ ] Inheritance (single, multiple, multilevel)
- [ ] Method Overriding
- [ ] Method Resolution Order (MRO)
- [ ] Polymorphism
- [ ] Abstraction & ABC module
- [ ] Dunder/Magic methods (`__str__`, `__repr__`, `__eq__`, etc.)
- [ ] Operator overloading
- [ ] Property decorators (`@property`)
- [ ] Dataclasses
- [ ] `__slots__` for memory optimization

PHASE 6: Intermediate & Advanced Python

File Handling

- [ ] Reading files (`open`, `with`)
- [ ] Writing & appending files
- [ ] Reading line by line
- [ ] Working with CSV (`csv` module)
- [ ] Working with JSON (`json` module)
- [ ] Working with XML
- [ ] Pickle (object serialization)
- [ ] Working with paths (`os.path`, `pathlib`)

Error Handling

- [ ] try / except / finally
- [ ] Multiple except blocks
- [ ] `raise` custom exceptions
- [ ] Custom exception classes
- [ ] Context managers (`with` statement)
- [ ] Creating context managers (`__enter__`, `__exit__`)

Iterators & Generators

- [ ] Iterator protocol (`__iter__`, `__next__`)
- [ ] Generator functions
- [ ] Generator expressions

- [ ] `itertools` module deep dive
- [ ] `functools` module (lru_cache, partial, reduce)

Modules & Packages

- [ ] Importing modules
- [ ] Creating modules
- [ ] `__name__ == "__main__"`
- [ ] Creating packages (`__init__.py`)
- [ ] Installing with pip
- [ ] Virtual environments (venv, virtualenv)
- [ ] `requirements.txt` & `pip freeze`
- [ ] Poetry / uv (modern package managers)

Concurrency & Parallelism

- [ ] Threading (`threading` module)
- [ ] Multiprocessing (`multiprocessing`)
- [ ] Async/Await (`asyncio`)
- [ ] `concurrent.futures`
- [ ] GIL (Global Interpreter Lock) understanding
- [ ] Locks, Semaphores, Queues

Advanced Features

- [ ] Type hints (`typing` module)
- [ ] Pydantic for data validation
- [ ] Memory management & garbage collection
- [ ] `__new__` vs `__init__`
- [ ] Metaclasses
- [ ] Descriptors
- [ ] Monkey patching
- [ ] Profiling (`cProfile`, `timeit`)

PHASE 7: Data Structures & Algorithms (Big Tech Interview Prep)

Arrays & Strings

- [ ] Two pointers technique
- [ ] Sliding window
- [ ] Prefix sum
- [ ] Kadane's algorithm
- [ ] Dutch national flag problem

- [ ] Trapping rain water
- [ ] Longest substring without repeating characters
- [ ] String matching (KMP, Rabin-Karp)

Linked Lists

- [ ] Singly linked list (implementation)
- [ ] Doubly linked list
- [ ] Circular linked list
- [ ] Reverse linked list
- [ ] Detect cycle (Floyd's algorithm)
- [ ] Merge two sorted lists
- [ ] LRU Cache implementation

Stacks & Queues

- [ ] Stack implementation
- [ ] Queue implementation
- [ ] Deque (`collections.deque`)
- [ ] Priority Queue / Heap (`heapq`)
- [ ] Monotonic stack
- [ ] Next greater element
- [ ] Valid parentheses

Trees

- [ ] Binary Tree implementation
- [ ] Tree traversals (inorder, preorder, postorder)
- [ ] Level order traversal (BFS)
- [ ] Binary Search Tree (BST)
- [ ] AVL Tree (basics)
- [ ] Tree height & diameter
- [ ] Lowest Common Ancestor (LCA)
- [ ] Serialize / Deserialize tree
- [ ] Trie (prefix tree)
- [ ] Segment tree (basics)
- [ ] Fenwick tree / BIT (basics)

Graphs

- [ ] Graph representation (adjacency list/matrix)
- [ ] BFS traversal
- [ ] DFS traversal
- [ ] Detect cycle in graph

- [ ] Topological sort
- [ ] Dijkstra's algorithm
- [ ] Bellman-Ford
- [ ] Floyd-Warshall
- [ ] Prim's & Kruskal's MST
- [ ] Union-Find (DSU)
- [ ] Strongly connected components

Dynamic Programming

- [ ] Memoization vs Tabulation
- [ ] Fibonacci DP
- [ ] 0/1 Knapsack
- [ ] Unbounded Knapsack
- [ ] Longest Common Subsequence
- [ ] Longest Increasing Subsequence
- [ ] Edit Distance
- [ ] Coin Change
- [ ] Matrix Chain Multiplication
- [ ] DP on trees
- [ ] DP on grids

Greedy & Backtracking

- [ ] Activity selection
- [ ] Huffman coding
- [ ] N-Queens
- [ ] Sudoku solver
- [ ] Permutations & Combinations
- [ ] Subset sum

Bit Manipulation

- [ ] AND, OR, XOR, NOT, shifts
- [ ] Check power of 2
- [ ] Count set bits
- [ ] Single number problems
- [ ] Bitmask DP

Math & Number Theory

- [ ] Sieve of Eratosthenes
- [ ] GCD/LCM (Euclidean algorithm)
- [ ] Modular arithmetic

- [ ] Fast exponentiation
- [ ] Prime factorization

Big O & Complexity Analysis

- [ ] Time complexity analysis
- [ ] Space complexity analysis
- [ ] Amortized analysis
- [ ] Recurrence relations & Master theorem

PHASE 8: NumPy (Numerical Computing)

- [ ] Installation & import conventions
- [ ] Creating arrays (`array`, `zeros`, `ones`, `arange`, `linspace`)
- [ ] Array attributes (shape, dtype, ndim, size)
- [ ] Indexing & slicing
- [ ] Boolean indexing & fancy indexing
- [ ] Reshaping (`reshape`, `flatten`, `ravel`)
- [ ] Stacking (`vstack`, `hstack`, `concatenate`)
- [ ] Splitting arrays
- [ ] Broadcasting rules
- [ ] Arithmetic operations
- [ ] Universal functions (ufuncs)
- [ ] Aggregations (`sum`, `mean`, `std`, `min`, `max`)
- [ ] Linear algebra (`np.linalg`)
- [ ] Random module (`np.random`)
- [ ] Saving/loading arrays (`np.save`, `np.load`)
- [ ] Performance vs Python lists

PHASE 9: Pandas (Data Analysis)

- [ ] Series creation & operations
- [ ] DataFrame creation
- [ ] Reading data (CSV, Excel, JSON, SQL, Parquet)
- [ ] Writing data
- [ ] Inspecting data (`head`, `tail`, `info`, `describe`)
- [ ] Selecting columns & rows
- [ ] `loc` vs `iloc`
- [ ] Filtering data (boolean indexing)
- [ ] Adding/removing columns

- [ ] Handling missing data (`isna`, `fillna`, `dropna`)
- [ ] Data types & conversions
- [ ] String operations (`.str` accessor)
- [ ] Date/time operations (`.dt` accessor)
- [ ] Sorting (`sort_values`, `sort_index`)
- [ ] GroupBy operations
- [ ] Aggregations & transformations
- [ ] Pivot tables & cross-tabs
- [ ] Merging & joining (`merge`, `join`, `concat`)
- [ ] Apply, map, applymap
- [ ] Window functions (`rolling`, `expanding`)
- [ ] MultiIndex / hierarchical indexing
- [ ] Performance optimization tips

PHASE 10: Data Visualization

Matplotlib

- [ ] Basic plot, scatter, bar
- [ ] Histograms & box plots
- [ ] Multiple subplots
- [ ] Customizing plots (labels, legend, colors)
- [ ] Saving figures

Seaborn

- [ ] Statistical plots (distplot, heatmap, pairplot)
- [ ] Categorical plots
- [ ] Themes & styling

Plotly (Interactive)

- [ ] Plotly Express basics
- [ ] Interactive charts
- [ ] Dashboards

PHASE 11: Web Scraping & Automation

Requests & BeautifulSoup

- [ ] HTTP basics (GET, POST, headers, cookies)
- [ ] `requests` library

- [ ] Parsing HTML with BeautifulSoup
- [ ] Selecting elements (CSS selectors, find, find_all)
- [ ] Handling pagination
- [ ] Sessions & authentication

Selenium

- [ ] Browser automation setup
- [ ] WebDriver basics
- [ ] Locating elements
- [ ] Interactions (click, type, scroll)
- [ ] Waits (implicit/explicit)
- [ ] Headless browsing

Playwright

- [ ] Installation & setup
- [ ] Sync vs Async API
- [ ] Page navigation & interactions
- [ ] Selectors (CSS, XPath, text)
- [ ] Auto-waiting
- [ ] Screenshots & PDFs
- [ ] Network interception
- [ ] Multiple browser contexts
- [ ] Headless vs headed mode
- [ ] Handling iframes & shadow DOM

Scrapy (Production-grade)

- [ ] Spider basics
- [ ] Items & pipelines
- [ ] Middlewares
- [ ] Scrapy shell

Automation

- [ ] PyAutoGUI for GUI automation
- [ ] `os` & `shutil` for file automation
- [ ] `schedule` for task scheduling
- [ ] Sending emails (`smtplib`)
- [ ] WhatsApp/Telegram bot APIs

PHASE 12: Databases with Python

- [ ] SQLite with `sqlite3`
- [ ] MySQL/PostgreSQL with connectors
- [ ] SQLAlchemy ORM basics
- [ ] CRUD operations
- [ ] MongoDB with `pymongo`
- [ ] Redis with `redis-py`
- [ ] Supabase Python client
- [ ] Database migrations (Alembic)

PHASE 13: APIs & Web Development

Building APIs

- [ ] FastAPI basics
- [ ] Path & query parameters
- [ ] Request body & Pydantic models
- [ ] Authentication (JWT, OAuth)
- [ ] Middleware
- [ ] Dependency injection
- [ ] Async endpoints
- [ ] Background tasks
- [ ] WebSockets
- [ ] Flask basics (alternative)
- [ ] Django basics (alternative)

Consuming APIs

- [ ] REST API calls with `requests`
- [ ] Async calls with `httpx` / `aiohttp`
- [ ] Working with JSON responses
- [ ] Error handling & retries
- [ ] Rate limiting

PHASE 14: Machine Learning

Scikit-learn

- [ ] ML workflow overview
- [ ] Train/test split

- [ ] Feature scaling (StandardScaler, MinMaxScaler)
- [ ] Encoding categoricals (OneHot, Label)
- [ ] Pipelines

Supervised Learning

- [ ] Linear Regression
- [ ] Logistic Regression
- [ ] K-Nearest Neighbors (KNN)
- [ ] Decision Trees
- [ ] Random Forest
- [ ] Gradient Boosting (XGBoost, LightGBM, CatBoost)
- [ ] Support Vector Machines (SVM)
- [ ] Naive Bayes

Unsupervised Learning

- [ ] K-Means Clustering
- [ ] Hierarchical Clustering
- [ ] DBSCAN
- [ ] PCA (Dimensionality Reduction)
- [ ] t-SNE / UMAP

Model Evaluation

- [ ] Accuracy, Precision, Recall, F1
- [ ] Confusion matrix
- [ ] ROC-AUC
- [ ] Cross-validation
- [ ] Hyperparameter tuning (GridSearch, RandomSearch)
- [ ] Bias-variance tradeoff
- [ ] Overfitting & regularization (L1, L2)

Feature Engineering

- [ ] Handling missing values
- [ ] Outlier detection
- [ ] Feature selection
- [ ] Feature creation
- [ ] Encoding strategies

PHASE 15: Deep Learning

PyTorch

- [ ] Tensors basics
- [ ] Tensor operations & broadcasting
- [ ] Autograd & gradient computation
- [ ] Building neural networks (`nn.Module`)
- [ ] Loss functions
- [ ] Optimizers (SGD, Adam, etc.)
- [ ] Training loop
- [ ] DataLoader & Dataset
- [ ] GPU/CUDA usage
- [ ] Saving & loading models
- [ ] Transfer learning

TensorFlow / Keras (alternative)

- [ ] Sequential & Functional API
- [ ] Building models
- [ ] Training & evaluation

Neural Network Architectures

- [ ] Feedforward NN (MLP)
- [ ] Convolutional NN (CNN)
- [ ] Recurrent NN (RNN, LSTM, GRU)
- [ ] Transformers (attention mechanism)
- [ ] Autoencoders
- [ ] GANs (basics)

Deep Learning Concepts

- [ ] Activation functions (ReLU, Sigmoid, Tanh, GELU)
- [ ] Backpropagation
- [ ] Batch normalization
- [ ] Dropout
- [ ] Learning rate scheduling
- [ ] Gradient clipping
- [ ] Weight initialization

PHASE 16: NLP & Computer Vision

NLP

- [ ] Tokenization, stemming, lemmatization
- [ ] NLTK & spaCy basics
- [ ] Bag of Words, TF-IDF
- [ ] Word embeddings (Word2Vec, GloVe)
- [ ] HuggingFace Transformers
- [ ] Fine-tuning BERT/GPT
- [ ] Text classification
- [ ] Named Entity Recognition (NER)

Computer Vision

- [ ] OpenCV basics
- [ ] Image processing (resize, filters, edge detection)
- [ ] Pillow (PIL)
- [ ] Image augmentation
- [ ] Object detection (YOLO basics)
- [ ] Image classification with CNNs

PHASE 17: LLMs & Generative AI

- [ ] OpenAI API
- [ ] Anthropic Claude API
- [ ] Google Gemini API
- [ ] Prompt engineering basics
- [ ] LangChain framework
- [ ] LlamaIndex
- [ ] RAG (Retrieval Augmented Generation)
- [ ] Vector databases (Pinecone, Chroma, Weaviate)
- [ ] Embeddings & similarity search
- [ ] Function calling / tool use
- [ ] Agents & multi-agent systems
- [ ] Streaming responses
- [ ] Token counting & cost optimization

PHASE 18: MLOps & Deployment

- [ ] Saving/loading ML models (pickle, joblib)

- [ ] Model serving with FastAPI
- [ ] Docker basics for Python
- [ ] Streamlit for ML demos
- [ ] Gradio for ML demos
- [ ] MLflow for experiment tracking
- [ ] Weights & Biases (wandb)
- [ ] Deploying to cloud (AWS, GCP, Azure basics)
- [ ] Hugging Face Spaces
- [ ] Vercel / Render for Python apps

PHASE 19: Testing & Code Quality

- [ ] `unittest` framework
- [ ] `pytest` framework
- [ ] Fixtures & parametrize
- [ ] Mocking (`unittest.mock`)
- [ ] Code coverage
- [ ] Type checking with `mypy`
- [ ] Linting (`flake8`, `pylint`, `ruff`)
- [ ] Formatting (`black`, `isort`)
- [ ] Pre-commit hooks
- [ ] Logging (`logging` module)

PHASE 20: Important Python Libraries (Quick Reference)

Standard Library Essentials

- [ ] `os`, `sys`, `pathlib`
- [ ] `datetime`, `time`, `calendar`
- [ ] `collections` (Counter, defaultdict, deque, OrderedDict)
- [ ] `itertools`, `functools`
- [ ] `re` (regex)
- [ ] `json`, `csv`, `pickle`
- [ ] `random`, `math`, `statistics`
- [ ] `argparse`, `sys.argv`
- [ ] `subprocess`
- [ ] `urllib`

Third-Party Must-Knows

- [ ] `requests` — HTTP
- [ ] `numpy` — Numerical
- [ ] `pandas` — Data
- [ ] `matplotlib` / `seaborn` — Plotting
- [ ] `scikit-learn` — ML
- [ ] `pytorch` / `tensorflow` — DL
- [ ] `transformers` — NLP/LLMs
- [ ] `fastapi` — APIs
- [ ] `pydantic` — Validation
- [ ] `sqlalchemy` — ORM
- [ ] `playwright` / `selenium` — Browser automation
- [ ] `beautifulsoup4` — HTML parsing
- [ ] `pytest` — Testing
- [ ] `rich` — Terminal output
- [ ] `tqdm` — Progress bars
- [ ] `python-dotenv` — Env vars
- [ ] `pillow` — Images
- [ ] `opencv-python` — CV
- [ ] `streamlit` — Quick UIs

PHASE 21: Big Tech Interview Prep (Python-Specific)

Coding Patterns (LeetCode-style)

- [ ] Two Pointers
- [ ] Sliding Window
- [ ] Fast & Slow Pointers
- [ ] Merge Intervals
- [ ] Cyclic Sort
- [ ] In-place Reversal of Linked List
- [ ] Tree BFS / DFS
- [ ] Two Heaps
- [ ] Subsets / Backtracking
- [ ] Modified Binary Search
- [ ] Top K Elements
- [ ] K-way Merge
- [ ] 0/1 Knapsack DP
- [ ] Topological Sort

System Design (Python perspective)

- [ ] Designing a URL shortener
- [ ] Designing a rate limiter
- [ ] Designing a cache (LRU/LFU)
- [ ] Designing a message queue
- [ ] Designing a notification system
- [ ] Database sharding & replication basics
- [ ] Microservices vs Monolith
- [ ] Caching strategies
- [ ] Load balancing concepts

Python Interview Trivia

- [ ] Mutable vs immutable types
- [ ] Shallow vs deep copy
- [ ] `is` vs `==`
- [ ] GIL deep dive
- [ ] How dict/set work internally (hashing)
- [ ] Memory model & reference counting
- [ ] Generator vs Iterator
- [ ] `*args` vs `**kwargs`
- [ ] Decorators & closures (in-depth)
- [ ] Context managers (in-depth)
- [ ] Metaclasses
- [ ] Class vs static vs instance methods
- [ ] Multiple inheritance & MRO
- [ ] `__slots__` use cases
- [ ] Common Python gotchas (mutable default args, late binding)

Behavioral & Resume Prep

- [ ] STAR method for behavioral questions
- [ ] Building a Python project portfolio
- [ ] GitHub README best practices
- [ ] Open-source contributions

PHASE 22: Capstone Projects (Apply Everything)

- [ ] CLI tool (with `argparse` / `click`)
- [ ] Web scraper (Playwright + Pandas)
- [ ] REST API (FastAPI + DB)

- [ ] ML model + deployment (Streamlit/Gradio)
- [ ] LLM-powered app (RAG with vector DB)
- [ ] Telegram/Discord bot
- [ ] Data analysis dashboard
- [ ] End-to-end DL project (CV or NLP)
- [ ] Automation script for real workflow
- [ ] Open-source contribution

> **Tip:** Don't rush phases. Build small projects after each phase to lock in concepts. For interviews, target ~150–200 LeetCode problems across the patterns above.